

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web 2

ENTREGA 2A

Cientes Supabase y Protección de Rutas

Proyecto: gestión-académica

Fase 1 — Fundación | Prerrequisito: Entregas 1A y 1B completadas

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Comprender cómo funciona la autenticación SSR con cookies en Supabase.
- Crear los clientes de Supabase diferenciados para browser y servidor.
- Configurar proxy.ts para proteger rutas que requieren autenticación.
- Implementar el callback de confirmación de email para completar el flujo de registro.

Prerrequisitos

- ✓ Entregas 1A y 1B completadas
- ✓ Dependencias @supabase/supabase-js y @supabase/ssr instaladas
- ✓ Variables de entorno configuradas en .env.local
- ✓ Tablas creadas en Supabase Dashboard

Teoría: Autenticación SSR con Supabase

Antes de escribir código, es fundamental entender cómo funciona la autenticación en una aplicación con Server-Side Rendering (SSR).

El problema: ¿cómo sabe el servidor quién es el usuario?

En una aplicación tradicional de frontend (SPA), el token de autenticación se guarda en localStorage del navegador. El problema es que localStorage no está disponible en el servidor. Cuando Next.js renderiza una página en el servidor, no puede leer localStorage.

La solución es usar cookies HTTP. Las cookies viajan automáticamente con cada petición HTTP, tanto del navegador al servidor como del servidor al navegador. Así, el token de sesión está disponible en ambos lados.

Flujo de autenticación SSR

FLUJO DE AUTENTICACIÓN SSR CON COOKIES:

1. Usuario hace login
 - ↳ Supabase Auth genera tokens (access + refresh)
 - ↳ Se guardan en cookies HTTP
2. Usuario visita una página protegida
 - ↳ El navegador envía las cookies automáticamente
 - ↳ proxy.ts intercepta la petición
 - ↳ Lee las cookies y verifica el token
 - ↳ Si es válido: deja pasar
 - ↳ Si NO es válido: redirige a /login
3. Token expirado

- ↳ proxy.ts detecta que expiró
- ↳ Usa el refresh token para obtener uno nuevo
- ↳ Actualiza las cookies automáticamente
- ↳ El usuario nunca nota la renovación

¿Por qué necesitamos dos clientes de Supabase?

Next.js ejecuta código en dos entornos diferentes, y cada uno maneja las cookies de forma distinta:

Entorno	Cliente	Cómo accede a cookies
Navegador	<code>createBrowserClient()</code>	Las cookies se manejan automáticamente por el navegador
Servidor	<code>createServerClient()</code>	Usa la función <code>cookies()</code> de Next.js para leer/escribir cookies

Regla crítica de Supabase

NUNCA uses los métodos individuales `get()`, `set()` o `remove()` para cookies. Supabase requiere SOLO `getAll()` y `setAll()`. Usar los métodos individuales causa errores de autenticación silenciosos que son muy difíciles de debuggear.

Crear el cliente de Supabase para el navegador

Este cliente se usa en componentes marcados con **"use client"** — componentes que se ejecutan en el navegador del usuario.

Crea el archivo **lib/supabase/client.ts**:

Archivo: lib/supabase/client.ts

```
import { createBrowserClient } from "@supabase/ssr"

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!
  )
}
```

Este es el cliente más simple. El navegador maneja las cookies automáticamente, así que no necesitamos configuración adicional.

¿Cómo se usa?

Ejemplo de uso (referencia)

```
// En cualquier componente "use client"
"use client"
import { createClient } from "@lib/supabase/client"

export function MiComponente() {
  const supabase = createClient()
```

```
// Ahora puedes usar: supabase.auth.signIn(), supabase.from()...
}
```

Nota: El operador !

El signo de exclamación (!) después de `process.env.NEXT_PUBLIC_SUPABASE_URL!` le dice a TypeScript: "confía en mí, esta variable existe". Esto es seguro porque configuramos `.env.local` en la Entrega 1A.

Crear el cliente de Supabase para el servidor

Este cliente se usa en Server Components, API Routes y `proxy.ts`. Es más complejo porque debe interactuar con las cookies de Next.js explícitamente.

Crea el archivo **lib/supabase/server.ts**:

```
Archivo: lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr"
import { cookies } from "next/headers"

export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet, _headers) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // El método setAll fue llamado desde un Server Component.
            // Esto se puede ignorar si proxy.ts está refrescando
            // las sesiones de usuario.
          }
        },
      },
    },
  )
}
```

Desglose línea por línea

Línea 2: `cookies` es una función asíncrona de Next.js que da acceso al almacén de cookies del request actual.

Línea 4: La función es **async** porque `cookies()` retorna una promesa en Next.js 16.

Líneas 11-13: `getAll()` lee todas las cookies del request. Supabase usa múltiples cookies para almacenar tokens.

Líneas 14-22: `setAll()` escribe las cookies actualizadas. El try/catch es necesario porque los Server Components no pueden escribir cookies (solo leerlas). `proxy.ts` se encarga de la escritura real.

Importante: `getAll` y `setAll`

Observa que SOLO usamos `getAll()` y `setAll()`. Nunca `get()`, `set()` o `remove()` individuales. Este es el patrón oficial de Supabase y usar otro patrón causará que la autenticación falle silenciosamente.

Crear `proxy.ts` para la protección de rutas

En Next.js 16, el archivo `proxy.ts` reemplaza al antiguo `middleware.ts`. Se ejecuta ANTES de cada petición y tiene tres responsabilidades críticas:

1. Refrescar el token de autenticación si ha expirado.
2. Sincronizar las cookies entre el request y la response.
3. Proteger rutas que requieren autenticación (redirigir a `/login` si no hay sesión).

Crea el archivo **`proxy.ts`** en la **raíz del proyecto** (al mismo nivel que `package.json`):

Archivo: `proxy.ts` (raíz del proyecto)

```
import { createServerClient } from "@supabase/ssr"
import { NextResponse, type NextRequest } from "next/server"

export async function proxy(request: NextRequest) {
  // 1. Crear una respuesta inicial que pasa el request original
  let supabaseResponse = NextResponse.next({
    request,
  })

  // 2. Crear cliente Supabase con manejo de cookies
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll()
        },
        setAll(cookiesToSet, headers) {
          // Actualizar cookies en el request (para Server Components)
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          )
          // Crear nueva respuesta con request actualizado
          supabaseResponse = NextResponse.next({
            request,
          })
          // Actualizar cookies en la respuesta (para el navegador)
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        }
      }
    }
  )
}
```

```

        // Sincronizar headers
        Object.entries(headers).forEach(([key, value]) =>
            supabaseResponse.headers.set(key, value)
        )
    },
},
}
)

// 3. IMPORTANTE: Verificar el usuario actual
// No ejecute código entre createServerClient y auth.getUser()
const {
    data: { user },
} = await supabase.auth.getUser()

// 4. Definir rutas públicas (no requieren autenticación)
const publicRoutes = ["/login", "/registro", "/auth"]
const isPublicRoute = publicRoutes.some((route) =>
    request.nextUrl.pathname.startsWith(route)
)

// 5. Si no hay usuario y la ruta NO es pública, redirigir a login
if (!user && !isPublicRoute && request.nextUrl.pathname !== "/") {
    const url = request.nextUrl.clone()
    url.pathname = "/login"
    return NextResponse.redirect(url)
}

// 6. Si hay usuario y está en una ruta de auth, redirigir al dashboard
if (user && isPublicRoute) {
    const url = request.nextUrl.clone()
    url.pathname = "/estudiante" // Ruta por defecto
    return NextResponse.redirect(url)
}

// 7. Retornar la respuesta con cookies actualizadas
return supabaseResponse
}

// Configuración: en qué rutas se ejecuta proxy.ts
export const config = {
    matcher: [
        /*
        * Ejecutar en todas las rutas EXCEPTO:
        * - _next/static (archivos estáticos de Next.js)
        * - _next/image (optimización de imágenes)
        * - favicon.ico (icono del sitio)
        * - Archivos de imagen (svg, png, jpg, etc.)
        */
        "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)"
    ],
}

```

Desglose de proxy.ts

Este archivo es el más complejo de la aplicación. Analicemos cada sección:

Sección 1 — Respuesta inicial: Creamos un `NextResponse.next()` que "pasa" el request al siguiente handler. Es la respuesta por defecto si todo está bien.

Sección 2 — Cliente con cookies: El cliente lee cookies del request entrante (`getAll`) y actualiza cookies tanto en el request como en la response (`setAll`). Esta sincronización bidireccional es crítica.

Sección 3 — `auth.getUser()`: Verifica si hay un usuario autenticado. Internamente, valida el token JWT y lo refresca si ha expirado. NUNCA pongas código entre `createServerClient` y `getUser()`.

Sección 4-6 — Lógica de rutas: Define qué rutas son públicas y redirige según el estado de autenticación.

Sección 7 — Retorno: Siempre retorna `supabaseResponse` para mantener las cookies sincronizadas.

Nunca hagas esto

No modifiques `supabaseResponse` creando un nuevo `NextResponse.next()` después de la lógica de Supabase. Si necesitas una response diferente, copia las cookies con:
`myResponse.cookies.setAll(supabaseResponse.cookies.getAll())`

Ruta	Usuario	Acción
<code>/login</code>	No autenticado	Mostrar página de login
<code>/login</code>	Autenticado	Redirigir a <code>/estudiante</code>
<code>/estudiante</code>	Autenticado	Mostrar dashboard
<code>/estudiante</code>	No autenticado	Redirigir a <code>/login</code>
<code>/api/materias</code>	Cualquiera	Pasar (la API verifica internamente)
<code>/_next/static/...</code>	—	No ejecuta <code>proxy.ts</code> (matcher lo excluye)

Crear el callback de confirmación de email

Cuando un usuario se registra con email y password, Supabase le envía un email de confirmación. Al hacer clic en el enlace, el usuario es redirigido a una URL de callback en nuestra aplicación. Necesitamos crear esa ruta.

Crea el archivo `app/(auth)/auth/callback/route.ts`:

```
Archivo: app/(auth)/auth/callback/route.ts
import { NextResponse } from "next/server"
import { createClient } from "@lib/supabase/server"

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url)
```

```
// Extraer el code del query string
const code = searchParams.get("code")

// Parámetro "next" opcional para redirección personalizada
const next = searchParams.get("next") ?? "/"

if (code) {
  const supabase = await createClient()

  // Intercambiar el code por una sesión
  const { error } = await supabase.auth.exchangeCodeForSession(code)

  if (!error) {
    // Redirigir al usuario a la página deseada
    return NextResponse.redirect(`${origin}${next}`)
  }
}

// Si algo falla, redirigir a una página de error
return NextResponse.redirect(`${origin}/login?error=callback_failed`)
}
```

¿Qué es `exchangeCodeForSession`?

Cuando el usuario hace clic en el enlace de confirmación, Supabase añade un parámetro `code` a la URL. La función `exchangeCodeForSession()` toma ese código temporal y lo intercambia por tokens de sesión reales que se guardan en cookies.

Flujo de confirmación de email

FLUJO COMPLETO DE CONFIRMACIÓN:

1. Usuario se registra con email
2. Supabase envía email con enlace:
`https://tuapp.com/auth/callback?code=abc123`
3. Usuario hace clic en el enlace
4. Next.js ejecuta esta `route.ts`
5. `exchangeCodeForSession("abc123")` → sesión válida
6. Se guardan las cookies de sesión
7. Redirect a la página principal

Configurar la plantilla de email en Supabase

Para que el flujo de confirmación funcione con SSR, necesitamos modificar la plantilla de email de Supabase para que use nuestro callback:

4. Abre Supabase Dashboard y ve a Authentication > Email Templates.
5. Selecciona la plantilla **Confirm signup**.
6. En el cuerpo del email, busca la línea con el enlace de confirmación y cámbiala:

Cambio en la plantilla de email


```

<!-- ANTES (por defecto): -->
<a href="{{ .ConfirmationURL }}">Confirm your email</a>

<!-- DESPUÉS (para SSR): -->
<a href="{{ .SiteURL }}/auth/callback?code={{ .TokenHash }}&type=email">
  Confirmar email
</a>

```

7. Haz clic en **Save**.

Importante: Site URL

Verifica que en Authentication > URL Configuration, el Site URL sea `http://localhost:3000` para desarrollo. En producción, cámbialo a tu dominio real.

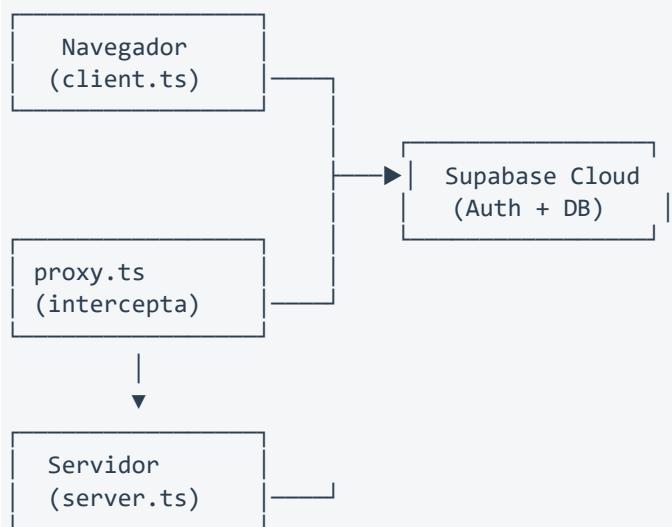
Resumen de archivos creados

En esta entrega creamos 3 archivos de infraestructura que serán la base de toda la autenticación:

Archivo	Responsabilidad
<code>lib/supabase/client.ts</code>	Cliente para componentes del navegador ("use client"). Manejo automático de cookies.
<code>lib/supabase/server.ts</code>	Cliente para Server Components, API Routes y Route Handlers. Usa <code>cookies()</code> de Next.js.
<code>proxy.ts</code>	Intercepta cada request. Refresca tokens, protege rutas, sincroniza cookies bidireccional.
<code>app/(auth)/auth/callback/route.ts</code>	Endpoint que recibe el código de confirmación de email y lo intercambia por una sesión.

Diagrama de arquitectura

ARQUITECTURA DE AUTENTICACIÓN:



Verificación Final

Confirma que todos estos puntos se cumplen antes de avanzar a la Entrega 2B:

- ✓ `lib/supabase/client.ts` existe y exporta `createClient()`
- ✓ `lib/supabase/server.ts` existe y exporta `createClient()` como `async`
- ✓ `proxy.ts` existe en la raíz del proyecto (NO dentro de `app/`)
- ✓ `app/(auth)/auth/callback/route.ts` existe y exporta `GET`
- ✓ La plantilla de email de Supabase apunta a `/auth/callback?code=...`
- ✓ El Site URL en Supabase está configurado como `http://localhost:3000`
- ✓ `npm run dev` inicia sin errores (el proxy no bloquea la página)

Tip: Cómo probar

En este punto aún no tenemos formularios de login/registro. Eso viene en la Entrega 2B. Lo importante ahora es que el servidor inicie sin errores y que la página de prueba (Card) siga visible en `localhost:3000`.

Ejercicio Propuesto

8. **Lee el código de `proxy.ts`:** Sin mirar las explicaciones de arriba, intenta describir con tus propias palabras qué hace cada sección numerada del archivo.
9. **Investiga `matcher`:** La expresión regular en `config.matcher` puede parecer intimidante. Investiga qué significa `(?!...)` en regex y por qué excluimos `_next/static`.
10. **Añade `logging`:** Agrega un `console.log` al inicio de `proxy.ts` que muestre la ruta actual y si hay usuario. Ejemplo: `console.log(`[proxy] ${request.nextUrl.pathname} - user: ${!!user}`)`. Navega por la app y observa los logs en la terminal.
11. **Pregunta de reflexión:** ¿Qué pasaría si alguien modifica las cookies en el navegador para hacerse pasar por otro usuario? ¿Supabase lo detectaría? (Pista: investiga qué hace `getUser()` internamente).

Próximo: Entrega 2B

En la siguiente entrega crearemos las páginas de login y registro con formularios funcionales usando `shadcn/ui`, la API Route para crear el perfil del usuario al registrarse, y la lógica de redirección por rol después del login.